

ECONOMICON

FinOps sin fricción, para quien no tiene tiempo de ser experto en FinOps.

Memoria del proyecto

Asistente FinOps para Azure — Propuesta

Alejandro Aguado · Paris Arcos · Lucía Mateo · Víctor Méndez

Trabajo Fin de Máster · Entrega prevista: octubre de 2026

Índice

A. Introducción y resumen ejecutivo 1

B. Caso de negocio 2

C. Contribución individual 4

D. Arquitectura técnica de la solución 6

E. Diseño y desarrollo de la solución de IA 10

F. Integración con herramientas DevOps 13

G. Uso de modelos preentrenados 15

H. Métricas y evaluación de la solución 16

I. Conclusiones y trabajo futuro 19

Referencias 20

A. Introducción y resumen ejecutivo

Gastar en la nube es fácil; entender el gasto, no. Las startups, los equipos pequeños y los desarrolladores independientes acumulan facturas de proveedores cloud sin visibilidad real de dónde se va el dinero. Rara vez tienen un equipo de FinOps, la práctica de dar visibilidad y control al coste de la nube, ni tiempo para analizar esas facturas a mano. Economicon nace para cerrar esa brecha: entender en qué se va el dinero de la nube no debería requerir un máster en facturas ni un equipo dedicado.

Esta memoria presenta Economicon, un asistente FinOps para personas que no son expertas en la materia. El alcance de este Trabajo Fin de Máster es Microsoft Azure. Sobre un conjunto de datos públicos de costes de Azure, servido por una interfaz de programación (API) simulada, la solución ingiere y normaliza el gasto, calcula indicadores que permiten actuar, detecta gasto anómalo y responde preguntas en lenguaje natural. Para responder, un agente conversacional usa generación aumentada por recuperación (RAG): antes de redactar, el sistema recupera los fragmentos de documentación y de datos que vienen al caso.

A.1. Resumen ejecutivo

- **Problema:** falta de visibilidad y de tiempo para analizar el gasto cloud en organizaciones sin función FinOps.
- **Solución:** un asistente que convierte datos de coste de Azure en indicadores claros, alertas de gasto anómalo y respuestas conversacionales con fuentes citadas.
- **Arquitectura:** aplicación web con backend en Python, procesamiento asíncrono de tareas y dos bases de datos, una relacional y otra vectorial, que se apoya en modelos de lenguaje a través de un gateway (un punto de entrada que gestiona el acceso a esos modelos). El detalle está en d.
- **Enfoque de inteligencia artificial (IA):** modelos preentrenados, sin entrenamiento propio. La adaptación al dominio se logra con RAG sobre los datos de coste y la documentación FinOps. Detalle en e y

g.

- **DevOps:** integración continua (CI) y despliegue continuo (CD), decisiones documentadas y cambios revisados mediante pull requests. Detalle en f.
- **Alcance del MVP:** el producto mínimo viable, es decir, la versión mínima que demuestra el circuito completo. Solo Azure, con datos públicos y sin conexión a la facturación real de ningún cliente.

Los resultados y sus límites se presentan en h. El circuito completo funciona de extremo a extremo con datos reproducibles. La utilidad para el público objetivo se valorará en un piloto con usuarios reales, como trabajo futuro.

B. Caso de negocio

B.1. La oportunidad

Economicon propone acercar FinOps a equipos sin especialistas mediante consultas en lenguaje natural. La demanda es una hipótesis que se contrastará con usuarios.

B.2. Audiencia objetivo

La propuesta se dirige a fundadores de startups, desarrolladores independientes y equipos pequeños que gestionan su propia infraestructura sin un equipo FinOps dedicado.

B.3. Análisis de mercado

Gartner prevé que el gasto mundial de los usuarios finales en nube pública alcance los 723.400 millones de dólares en 2025, frente a los 595.700 millones de 2024. Según Flexera, que encuestó en 2025 a 759 profesionales cloud, el 84 % sitúa la gestión del gasto entre sus principales retos y estima en un 27 % el gasto en infraestructura y plataformas en la nube (infraestructura y plataforma como servicio, IaaS y PaaS) que se desperdicia. [1, 2]

Estos datos justifican investigar el control del gasto. Economicon compite por facilidad de interpretación frente a los paneles nativos del proveedor.

B.4. Justificación de la solución

El asistente busca responder «¿en qué se me va el dinero?» con datos y fuentes. Combina cálculos deterministas de indicadores, anomalías y presupuestos con recuperación documental y generación de explicaciones. Los cálculos del servidor son verificables; la calidad de las respuestas generadas se evalúa en el apartado H. Se eligió Azure por disponer de datos públicos y un esquema de costes documentado, evitando usar facturas privadas de clientes. [3]

B.5. Viabilidad

Técnica: El prototipo implementa ingesta, normalización, indicadores, presupuestos y conversación documental. Las pruebas automáticas y los recorridos con datos reproducibles confirman que el circuito completo funciona de extremo a extremo; la calidad de las respuestas se mide en el apartado H. [3]

Operativa: Docker Compose permite iniciar el entorno local: once servicios permanentes, incluida la página pública, y dos de inicialización, más dos opcionales para IA. Una API simulada sirve datos públicos de Microsoft FinOps Toolkit con licencia MIT. [4]

Económica: La tabla B.1 compara dos escenarios de un año. El modelo cobra un pago único de despliegue de 200 € por cliente y una mensualidad de 50 €, con todos los clientes activos durante el año. Los precios son los que ha calculado el equipo. El coste fijo es de 3.200 € mensuales: 1.200 € de infraestructura, cifra que se toma como hipótesis, y 80 horas de mantenimiento y evolución a 25 € por hora.

Tabla B.1. Escenarios económicos a un año, en euros

Concepto	Conservador (20 clientes)	Optimista (100 clientes)
Ingresos por despliegue (200 € por cliente)	4.000	20.000
Ingresos por mensualidad (50 € durante 12 meses)	12.000	60.000
Costes (3.200 € al mes)	38.400	38.400
Resultado	**−22.400**	**41.600**

En el escenario conservador no se cubren los costes; en el optimista sí. El equilibrio recurrente requiere 64 clientes o 48 en el primer año contando el pago de despliegue. Si el cliente recuperara un tercio del desperdicio medio que cita Flexera, ahorraría el 9 % de su gasto: a partir de unos 556 € mensuales, una mensualidad de 50 € se compensa.

B.6. Impacto esperado

Se pretende facilitar la interpretación de costes sin formación especializada. La evaluación técnica mide acierto, latencia y trazabilidad de cifras, con métodos y objetivos descritos en el apartado H. La primera línea base, una plantilla sin modelo, no aprobó ninguno de los veinte casos con respuesta. El impacto empresarial requerirá un piloto que mida uso, tiempo ahorrado y beneficio económico. [5]

B.7. Costes y supuestos

Las mediciones de IA se separan del presupuesto de infraestructura.

Coste de la inteligencia artificial: La tabla G.1 distingue llamadas al modelo y operaciones completas: cada intento de chat tiene dos llamadas y puede reintentarse. La factura contrasta llamadas; el coste por operación procede de otra medición. [6] El registro del gateway situó en 0,0118 USD el coste acumulado de tres tandas diagnósticas y una comprobación de embeddings; esa cifra no es el gasto total del desarrollo. [5, 7]

Coste de infraestructura: El presupuesto de 1.200 € mensuales supone operación continua, sin IVA ni capa gratuita. Los modelos de lenguaje son un coste variable aparte, que crece con el uso.

B.8. Riesgos

Los riesgos técnicos principales son la calidad de las respuestas y la latencia. Se controlan con casos reproducibles, límites de generación y registro de fallos.

En lo económico, los reintentos y el volumen pueden elevar el gasto. Los límites por clave y el registro de consumo permiten controlarlo. [7]

Para reducir el riesgo de plazo se priorizaron los contenidos obligatorios, las entregas pequeñas revisables y la identificación explícita de los pendientes.

C. Contribución individual

¿Quién ha hecho qué en Economicon? Un trabajo en equipo solo se evalúa de forma individual si queda claro qué construyó cada persona. Este apartado muestra lo que aportó cada miembro, cómo rotaron los cuatro roles y hasta dónde llega la evidencia. Los roles —liderazgo, pairing (colaboración en la implementación, con coautoría), revisión y validación— rotan en cada historia de trabajo para que el conocimiento no se concentre y nadie revise ni valide su propio trabajo. Cada pull request declara quién ejerce cada rol y un registro reproducible contrasta esas declaraciones con el tablero de Trello y con la actividad en GitHub. [8]

C.1. Qué ha construido cada persona

La tabla C.1 resume la aportación principal de cada miembro; los párrafos siguientes la detallan.

Tabla C.1. Aportación principal de cada miembro.

Miembro	Aportación principal
Alejandro Aguado	Base de datos de costes y entrega continua: ingesta y normalización de los costes de Azure, contrato del asistente de costes y despliegue automático.
Víctor Méndez	Interfaz web: migración a TypeScript, sesión y estilos, validación en navegador real; explicación de anomalías y panel de evaluación.
Lucía Mateo	Observabilidad y evaluación: métricas, recuperación semántica, anomalías de coste y evaluación del asistente.
Paris Arcos	Inteligencia artificial y seguridad: recuperación documental en tres fases, pasarela de modelos, autenticación y aislamiento por cliente.

Alejandro Aguado construyó la base de datos de costes y la entrega continua, es decir, el despliegue automático de cada versión integrada: normalizó y cargó los costes de Azure, definió el contrato y las salvaguardas del asistente, empaquetó los servicios en contenedores Docker y montó el despliegue en un servidor privado. Sobre esa base añadió las consultas de gasto y de propiedad con evidencia y citas, los presupuestos, la cobertura de etiquetas, la selección de proveedor y modelos, la demostración reproducible, la integración del chat y la canalización de AWS.

Víctor Méndez llevó la migración de la interfaz web a TypeScript desde el inventario hasta el cierre del proyecto y con ella el sistema de estilos, el aviso de sesión caducada y la comprobación del recorrido completo en un navegador real. En el asistente aportó la explicación de anomalías, el panel de Grafana con los resultados de la evaluación, los reintentos acotados de la cola de ingesta, la etiqueta del selector de tema y los avisos de limitaciones de anomalías en español. También elaboró el registro de contribuciones en el que se apoya este apartado.

Lucía Mateo construyó la observabilidad, es decir, la capacidad de medir y vigilar el sistema (registro con correlación, métricas, trazabilidad y alertas), el entorno local con Docker Compose y la recuperación semántica del asistente. Es autora de la detección de anomalías de coste y de su panel, de los datos sintéticos y la demostración, de la marca y el español en la interfaz y de la evaluación del asistente: métricas, batería de referencia, robustez ante datos incompletos y reevaluaciones tras cada cambio, con los indicadores medibles para esta memoria y la reducción de los cortes de respuesta del chat. También definió los cuatro roles y el flujo de revisión.

Paris Arcos construyó la recuperación documental en tres fases —ingesta de archivos, fragmentación determinista y recuperación híbrida—, la integración de la pasarela que conecta con los modelos de lenguaje (LiteLLM y OpenRouter) y el modelo generativo único. En el producto aportó los indicadores de coste y el panel ejecutivo, el panel de salud, la presentación de la evaluación en Grafana, la autenticación de demostración con aislamiento por cliente, la gestión de secretos, la integración continua y la página

pública de presentación.

C.2. Roles y propuestas de cambio

El tablero del corte del 23 de octubre comprende 126 historias de trabajo, 79 de ellas ya en Hecho, es decir, 504 posiciones de rol posibles. Se reconocen 473; las 31 restantes no nombran a una persona, por lo que se dejan como huecos y no se reparten. La tabla C.2 recoge las asignaciones por rol. [9]

Tabla C.2. Asignaciones de rol por miembro en el corte del 23 de octubre. Una asignación es planificación, no participación.

Rol	Qué cuenta	Alejandro Aguado	Víctor Méndez	Lucía Mateo	Paris Arcos	Sin resolver
Liderazgo	Responde de la historia: alcance, implementación y cierre	30	30	32	27	7
Pairing	Colabora en la implementación y cuestiona las decisiones	36	26	27	29	8
Revisión	Lee el cambio y comprueba que es correcto y mantenible, sin ser su autor	21	31	36	30	8
Validación	Comprueba con pruebas y evidencia que el resultado cumple los criterios de aceptación	32	30	26	30	8
Total		**119**	**117**	**121**	**116**	**31**

El reparto de la tabla no es uniforme: el tablero fija como objetivo entre 22 y 28 asignaciones por persona y rol; solo cuatro de las dieciséis celdas lo cumplen. Es un margen de mejora del equilibrio, que la tabla refleja tal como está planificado.

La tabla C.3 cuenta las propuestas de cambio integradas por cada miembro, es decir, las abiertas por esa persona y fusionadas en la rama principal con cualquier método de integración. [10]

Tabla C.3. Propuestas de cambio integradas por miembro hasta el corte del 23 de octubre.

Miembro	Propuestas integradas
Alejandro Aguado	59
Víctor Méndez	18
Lucía Mateo	39
Paris Arcos	16
Total	**132**

C.3. Límites de la evidencia

Estas cifras se leen con prudencia. Una asignación de rol es planificación y no participación, por eso la tabla C.3 recoge lo efectivamente integrado. La autoría de una propuesta no equivale a trabajo en solitario, pero el pairing solo puede contrastarse donde se declaró: 29 de los 200 commits de la rama principal en el corte llevan una línea de coautoría que identifica a un miembro del equipo; el resto del pairing consta en notas del tablero que el registro no importa. Siete historias del cierre del MVP se cerraron por excepción de plazo, sin pairing ni aprobación humana, y no se atribuye a nadie trabajo ni

aprobaciones que no constan. Los resultados de las comprobaciones automáticas y los archivos de pruebas y documentación son evidencia del equipo, no de una persona. [8]

D. Arquitectura técnica de la solución

Economicon es una aplicación web que responde preguntas sobre los costes de Azure y su arquitectura se rige por dos criterios. El primero es que cada cifra mostrada la calcule código determinista y que el modelo de lenguaje solo redacte sobre evidencia ya verificada. El segundo es que cada cliente acceda únicamente a sus propios datos. Con esos criterios se organizan las capas, el camino de una pregunta y la entrada de datos. La misma arquitectura se ofrece en dos formas de despliegue: un entorno local con Docker Compose y una alternativa en la nube sobre Amazon Web Services (AWS). [3]

D.1. Capas y tecnologías

Economicon se organiza en cinco capas —interfaz, API, cola y procesador, datos e inteligencia artificial— con la observabilidad como capa transversal (figura D.1). La interfaz web muestra costes, anomalías y el asistente. La API autentica a cada persona, separa los datos de cada cliente y calcula costes y anomalías; en cada petición comprueba la sesión y que la persona pertenece al cliente indicado, de modo que enviar el identificador de un cliente no concede acceso por sí solo. La cola de mensajes y el procesador ejecutan en segundo plano la ingesta de costes y de documentos; los datos se reparten entre una base transaccional, un índice de similitud y un almacén de archivos. La inteligencia artificial pasa por una pasarela que concentra claves y límites de gasto antes del proveedor; Prometheus y Grafana observan el conjunto. [3]

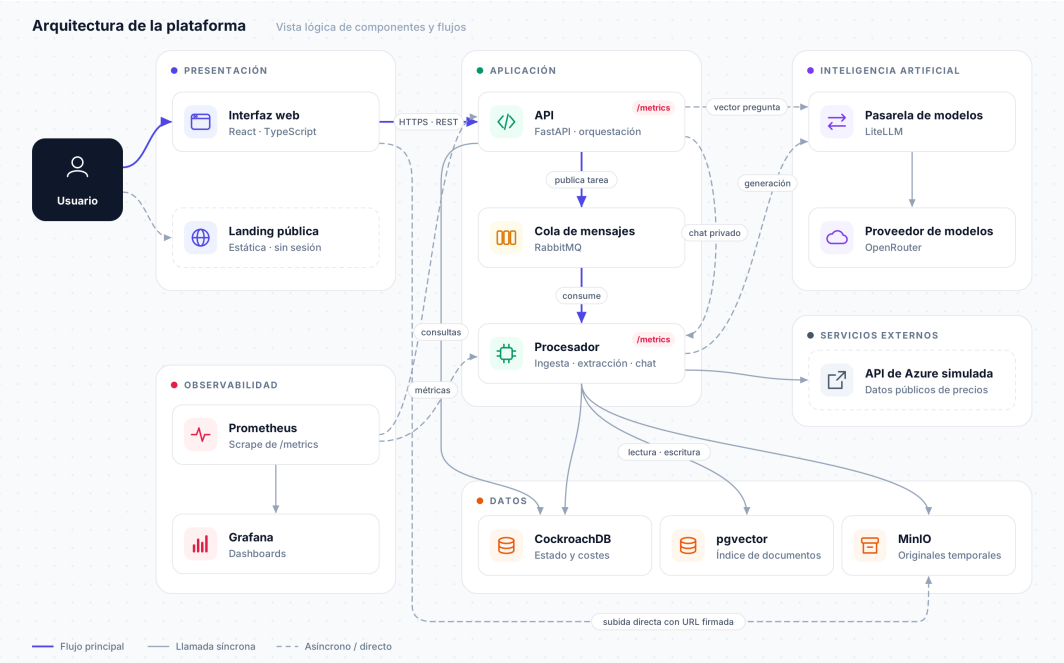


Figura D.1. Componentes del sistema. Las líneas continuas son el flujo principal; las discontinuas, comunicaciones laterales o que dependen de la configuración.

La tabla D.1 indica para qué se usa cada tecnología y por qué se eligió frente a la alternativa más habitual. [7]

D.2. Flujos de datos

Ambos flujos operan por cliente: la búsqueda filtra por cliente y los costes se guardan por cliente.

Tabla D.1. Tecnologías del sistema, su función y el motivo de su elección.

Tecnología	Para qué	Por qué esta y no otra
React y TypeScript	Interfaz web	El tipado estático detecta en compilación los desajustes con la API; frente a una interfaz generada en el servidor, admite el estado interactivo de las conversaciones en flujo.
FastAPI (Python)	API y procesador	Sus esquemas tipados validan cada entrada y generan el contrato de la API; Python comparte con el procesador las librerías de datos y de inteligencia artificial.
RabbitMQ	Cola de trabajos	Entrega con confirmación y cola duradera: la ingesta continúa aunque el procesador se reinicie. Frente a una cola en la base de datos, separa carga y fallos; a cambio, hay que tolerar mensajes duplicados.
CockroachDB	Estado: usuarios, trabajos, conversaciones y costes	Habla SQL, compatible con PostgreSQL, de modo que la aplicación puede funcionar sobre PostgreSQL, como prevé la alternativa en la nube. No se afirma que lo supere.
pgvector	Búsqueda por similitud de fragmentos	Extensión de PostgreSQL: un único motor filtra a la vez por cliente y por distancia, sin servicio vectorial dedicado. Calcula la distancia exacta, sin índice aproximado.
LiteLLM y OpenRouter	Acceso a los modelos	Concentran en un punto las claves, el techo de gasto y la política de privacidad y permiten cambiar de modelo sin tocar el código. El techo, de 10 USD al mes, es el presupuesto máximo de desarrollo y se aplica mediante una clave virtual de la pasarela.
MinIO	Almacén temporal de archivos subidos	Habla el protocolo S3 y mantiene los binarios fuera de la cola y de la base de datos.
Prometheus y Grafana	Métricas, alertas y paneles	Estándares abiertos de observabilidad; Grafana presenta también los resultados de la evaluación.
Docker Compose	Despliegue reproducible	Una orden levanta todo el sistema en un portátil o un servidor; no equivale a un orquestador de producción.

El camino de una pregunta

El asistente atiende cada pregunta por uno de dos caminos. Las consultas estructuradas —gasto por dimensión, propiedad de los costes y explicación de una anomalía— las resuelve el servidor con consultas a la base de datos y devuelve cifras exactas con su evidencia, sin intervención del modelo. Las preguntas abiertas recorren el chat con modelo de lenguaje. [4]

En el chat con modelo (figura D.2), la API abre una operación asociada a la conversación y la envía al procesador por una red privada y con una firma que solo conocen ambos; el procesador comprueba en cada paso que la operación sigue siendo de esa persona. El servidor prepara una única búsqueda documental por intento, sin que la decida el modelo, que redacta con un esquema fijo; el detalle de la recuperación y de la generación está en el apartado E.

La respuesta pasa por dos controles de validación, esquema y evidencia, que se detallan en el apartado E.4. La calidad y la latencia se miden en h.

Entrada de datos

Los costes de Azure se cargan con una herramienta del procesador que consulta una API de Azure simulada, con un conjunto público porque no hay cuenta real, los normaliza a importes decimales exactos y los guarda por cliente. Los documentos entran como texto o como archivo (Markdown, TXT, DOCX, DOC y PDF, digital o escaneado con reconocimiento óptico de caracteres local). El navegador sube el original a MinIO con una dirección firmada de vida corta; el procesador extrae su texto en un entorno aislado, lo divide en fragmentos estructurales y guarda en pgvector un vector por fragmento, calculado con text-embedding-3-small. El original se elimina cuando el texto está a salvo. [3, 11]

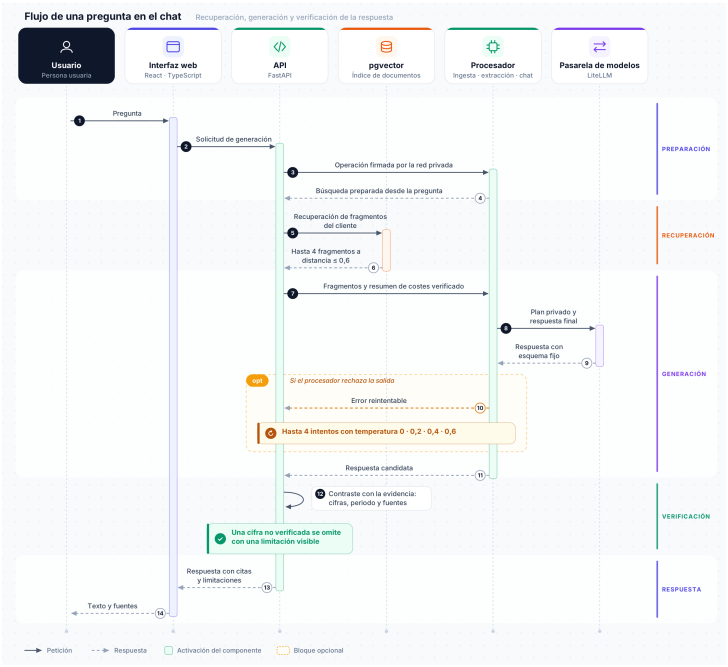


Figura D.2. Camino de una pregunta en el chat con modelo, con sus dos controles de validación.

D.3. Dos formas de despliegue

Economicon se despliega de dos formas que comparten la misma arquitectura por capas y cada una responde a un fin distinto. El entorno local, con Docker Compose, levanta el sistema completo con una orden y con datos simulados, sin credenciales de proveedor, y es la base de desarrollo y demostración. La alternativa en la nube traslada la misma arquitectura a servicios gestionados de AWS, que aportan copias de seguridad y alta disponibilidad, un proveedor de identidad externo y gestión de secretos.

Entorno local

Docker Compose levanta 11 servicios permanentes, incluida la página pública, y 2 tareas de un solo uso, que migran la base de datos y preparan el almacén de archivos, con 6 volúmenes persistentes que conservan los datos tras un reinicio; el perfil de inteligencia artificial añade la pasarela y su base de datos, con un volumen más. Prometheus recoge las métricas cada 15 s y Grafana alerta de los fallos de ingesta. [4]

Alternativa en la nube: AWS

La figura D.3 resume la alternativa preparada en AWS. Route 53 (DNS), CloudFront, un cortafuegos web (WAF) y un certificado (ACM) protegen el acceso; la interfaz reside en S3 privado y Cognito gestiona la identidad con OIDC. El balanceador interno dirige la API y Grafana a tareas de ECS Fargate en una red privada virtual (VPC). Allí se preparan también el procesador, el simulador Azure, Amazon MQ y dos bases RDS PostgreSQL. Prometheus y Grafana comparten una tarea privada; Grafana guarda sus metadatos y lee evaluaciones en RDS. CloudWatch recoge logs y señales de infraestructura. [12]

La tabla D.2 recoge qué sustituye a cada pieza local y por qué.

Las bases y el broker solo admiten conexiones de los grupos de seguridad autorizados. La referencia Terraform tiene 14 pruebas con proveedores simulados y la preparación inicial del controlador, otra; no crean infraestructura real. La CI prueba además el enrutamiento de CloudFront y la monitorización en contenedores aislados. El repositorio prepara un ciclo manual de creación y retirada y admite PostgreSQL, Cognito y Bedrock por configuración. El despliegue en una cuenta real de AWS **[Pendiente]**. [12, 13]

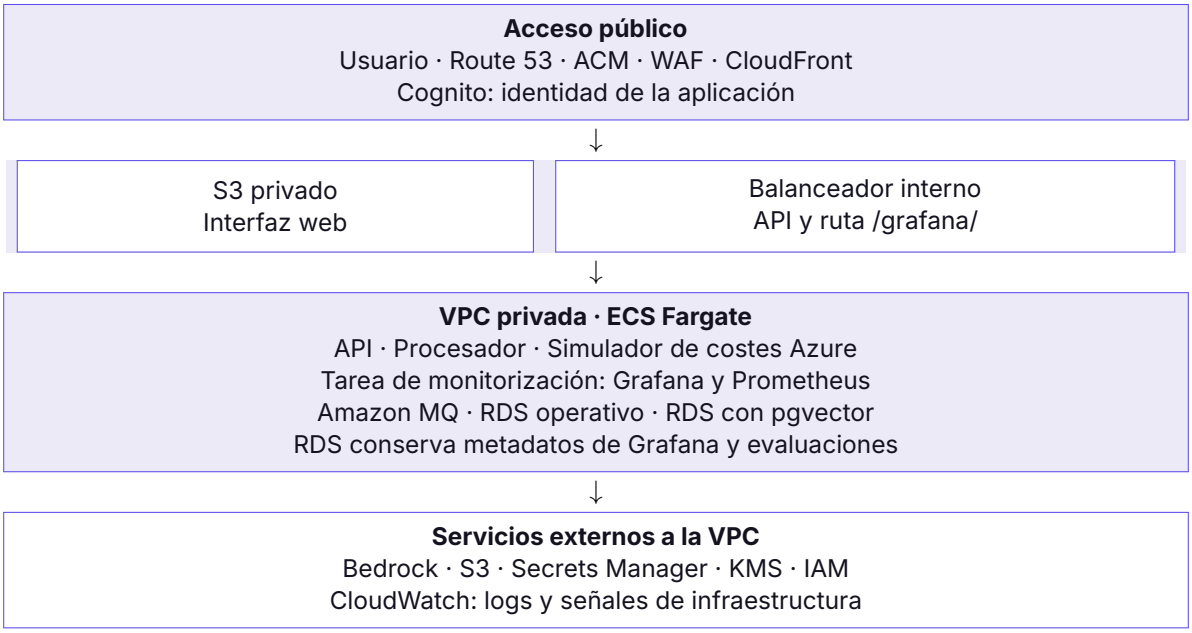


Figura D.3. Arquitectura preparada en AWS: acceso, servicios privados y observabilidad. Las flechas agrupan flujos; no representan un despliegue ejecutado.

Tabla D.2. Equivalencias entre la versión local y la alternativa en AWS.

En la versión local	En AWS	Por qué este cambio
Interfaz servida con la vista previa de Vite	S3 privado y CloudFront	Con certificado y sin servidor propio; CloudFront es el único acceso al almacén.
API, procesador y simulador en Compose	ECS Fargate y balanceador interno	Sin gestionar servidores ni direcciones públicas; el balanceador solo es alcanzable desde CloudFront.
RabbitMQ	Amazon MQ para RabbitMQ	Conserva el protocolo: productores y consumidores no cambian, a diferencia de la cola SQS de AWS.
CockroachDB y pgvector	Dos bases RDS PostgreSQL	Copias de seguridad y alta disponibilidad gestionadas; separar la base vectorial de la operativa aísla su consumo, a mayor coste. Exige adaptar el SQL propio de CockroachDB.
LiteLLM y OpenRouter	Amazon Bedrock	Modelos invocados dentro de la cuenta con permisos IAM; cambiar de modelo obliga a repetir la evaluación y reindexar todo el corpus.
Autenticación de demostración	Amazon Cognito	Proveedor de identidad externo; la API sigue resolviendo los clientes autorizados de cada persona.
MinIO	S3	Mismo protocolo, sin servidor que mantener.
Prometheus y Grafana	ECS Fargate y CloudWatch	Conserva paneles y métricas; Grafana usa RDS para metadatos y evaluaciones. CloudWatch recoge logs y señales de infraestructura.
Secretos de Compose	Secrets Manager, KMS e IAM	Claves cifradas y un rol de mínimo privilegio por servicio.

D.4. Alcance y límites

El sistema es una demostración con datos públicos y su margen de mejora se concentra en tres áreas. En seguridad, CockroachDB se ejecuta sin cifrado ni credenciales, válido solo para una demostración local y con la seguridad por activar en producción; la autenticación es de demostración, sin verificación en dos pasos, revocación de sesiones ni proveedor de identidad externo, que en la alternativa en la nube asume Cognito. En operación, Grafana alerta de los fallos de ingesta, pero no hay un receptor externo, como el correo, que los notifique; todo corre en una máquina, sin alta disponibilidad, copia de seguridad del conjunto ni transacción única entre base de datos, índice y cola, por lo que se toleran reintentos y duplicados. En despliegue, la interfaz se sirve con la vista previa de Vite y no con un servidor de producción; la alternativa en AWS es una arquitectura de referencia cuya aceptación es un objetivo por alcanzar.

E. Diseño y desarrollo de la solución de IA

Una respuesta sobre costes solo sirve para decidir gasto si se puede comprobar. Este apartado explica qué conocimiento prepara Economicon, cómo lo recupera y qué controles aplica antes de mostrar una respuesta. La tabla E.1 resume las decisiones; su calidad se evalúa en el apartado H. [3, 5, 14-16]

E.1. El modelo redacta sobre evidencia

Un modelo de lenguaje no conoce los datos de un cliente. Por eso Economicon recupera antes de redactar la información autorizada del cliente y la incorpora a la petición, de modo que la respuesta pueda citar sus fuentes en lugar de depender de la memoria del modelo; es el enfoque RAG descrito en el apartado A. Las preguntas sobre costes se resuelven en el servidor, sin modelo (apartado D); aquí se tratan las abiertas, que reciben el resumen de costes verificado y los pasajes recuperados (figura E.1). [15]

Tabla E.1. Decisiones de preparación, recuperación y generación

Elemento	Elección	Qué significa
Representación del texto	text-embedding-3-small; 1.536 dimensiones	Un embedding es un vector numérico (aquí, de 1.536 números) que representa el significado de un texto; permite comparar cada pasaje con la pregunta. El código admite además Titan de AWS, apagado por defecto.
Fragmentación	Objetivo de 600 tokens; máximo de 800; solapamiento del 10 %	Un token es la unidad en que el modelo divide el texto, aproximadamente una palabra o parte de ella. El corte respeta secciones, listas y tablas cuando caben.
Recuperación del chat	4 pasajes primarios; umbral coseno 0,6; hasta 20 candidatos	La distancia mide similitud (0 es idéntico). El umbral filtra la selección semántica; vecinos y pasajes prioritarios amplían el contexto, acotado también por tokens.
Recuperación híbrida (variante D)	Vectorial y por palabras, fusionadas por posición	Opción elegida tras comparar cuatro variantes.
Reintentos de una respuesta rechazada	Hasta 4 intentos; temperatura 0; 0,2; 0,4; 0,6	La temperatura controla la variabilidad de la respuesta (0 es la más determinista) y sube en cada reintento. Incluido en la versión final.



Figura E.1. Preparación del conocimiento y ciclo de una respuesta.

E.2. Del documento al vector

La extracción de texto y el reconocimiento óptico de caracteres de los ficheros escaneados se hacen dentro del sistema, aislados y con límites de páginas, tiempo y memoria. El original se retira cuando el texto, los vectores y el estado del trabajo ya están guardados, por lo que la recuperación y las citas no dependen del binario. [3]

Un fragmento (chunk) es la unidad de texto que se guarda y se recupera por separado. Se descartó cortar por un número fijo de caracteres porque no garantiza el máximo de tokens ni conserva las relaciones que permiten recuperar y citar una tabla, una lista o una subsección; por eso el algoritmo conserva secciones, listas, tablas y bloques de código cuando caben en el presupuesto de la tabla E.1 y aplica cortes reproducibles cuando no, sin usar modelos para decidir dónde cortar, lo que evita coste adicional y cortes distintos entre ejecuciones. El solapamiento queda registrado, de forma que el texto normalizado se puede reconstruir. [15]

Cada fragmento se representa como un vector (embedding) guardado en pgvector, con ranking exacto por distancia y filtrado siempre por cliente. Se eligió pgvector para reutilizar el almacenamiento y las transacciones existentes sin añadir un servicio vectorial; se descartó un índice aproximado global porque filtrar por cliente después de una selección aproximada puede dejar sin resultados. No se afirma que supere a otros motores. Documentos y preguntas deben compartir el mismo espacio de representación —modelo, proveedor, dimensión y preparación del texto—; cambiar de proveedor exige reindexar, porque coincidir en dimensión no garantiza compatibilidad semántica. Los metadatos que acompañan al vector (origen, título y procedencia de cada tramo) sirven para citar y reconstruir el texto; a la representación solo se incorporan el título y la sección, en las variantes C y D del apartado E.3.

E.3. Recuperación

La selección primaria y el umbral de la tabla E.1 se calibraron con preguntas etiquetadas por sección; pueden sobreajustarse a un corpus pequeño. El chat amplía esa selección con vecinos contiguos y prioridad de políticas: el umbral semántico no limita todos los pasajes añadidos. Los presupuestos de evidencia y tokens acotan el contexto; cuatro no es un límite global. Las campañas históricas conservan sus configuraciones. [17]

Para elegir la recuperación se compararon cuatro variantes acumulativas: A usaba fragmentos por caracteres y búsqueda vectorial; B introdujo fragmentación estructural; C añadió título y sección; D sumó búsqueda por palabras para nombres, magnitudes e identificadores. D fusiona ambas listas con Reciprocal Rank Fusion (RRF), que puntúa posiciones sin mezclar escalas. [7, 16]

Elegimos D porque respondió todas las preguntas de su campaña y seleccionó contexto más útil en las específicas; A y B cubrieron mejor algunas originales. Es una decisión ligada a esa muestra. D exige

índices preparados y activación explícita; el entorno de ejemplo usa embeddings simulados y mantiene el chat y las opciones RAG verificadas apagados. [4]

E.4. Generación y validación

El chat generativo se activa de forma explícita. Cuando está activo, el modelo no decide si se busca ni qué se busca, para que la evidencia que recibe no dependa de su criterio: el servidor prepara una única búsqueda por intento y la ejecuta antes del primer borrador. El modelo, cuya elección se justifica en el apartado G, trabaja en dos llamadas, un plan privado de afirmaciones y fuentes y una respuesta final contrastada con ese plan, con un esquema fijo —campos con tipos y límites definidos— y un prompt versionado, es decir, unas instrucciones al modelo con control de versiones. Recibe el contexto autorizado: pregunta, resumen de conversación, búsqueda, resumen de costes y captura de pantalla cuando existe; no decide ni ejecuta consultas adicionales. [3]

El prompt fija cuatro reglas. Los pasajes recuperados, el resumen de la conversación y la pregunta se tratan como datos y no como instrucciones, lo que limita la inyección de órdenes a través de un documento. No se inventan cifras, fuentes ni causas, y las métricas financieras solo copian los registros que el servidor autoriza. Si faltan datos, el modelo declara con un estado explícito qué falta en lugar de rellenar. Los cálculos con cifras del usuario se presentan como hipótesis no verificadas. Las cifras de pantalla tienen procedencia propia, permiten cálculos citables y no dan autoridad financiera. Un prompt reduce el riesgo, pero no lo garantiza, por lo que el control real es el del servidor. [18]

La respuesta pasa por dos controles que verifican campos, referencias y operaciones concretas, sin garantizar la corrección semántica de toda ella. El primero exige que la salida cumpla el esquema; si la rechaza, la API repite la generación con una temperatura creciente, hasta el máximo de intentos de la tabla E.1, de modo que un reintento no repita la misma salida. El segundo existe porque un modelo puede redactar cifras verosímiles que no coinciden con los datos; contrasta la respuesta con la evidencia autorizada: cada cifra financiera debe coincidir con la consulta de costes del servidor, y la que no coincide se omite y se sustituye por una limitación visible. Otras incoherencias, como una fuente desconocida o un periodo distinto del consultado, terminan la operación como fallida sin mostrar la respuesta. [19]

La conversación se recupera tras un fallo o una cancelación, y el historial largo se resume para acotar el contexto del modelo. Las consultas de propiedad, gasto y anomalías aún no están disponibles como herramientas que el modelo pueda invocar; conectarlas al chat es el siguiente paso del diseño (apartado I).

F. Integración con herramientas DevOps

Este apartado describe las herramientas DevOps de Economicon. La tabla F.1 relaciona cada una con sus evidencias y límites. [3, 4, 8, 12]

F.1. Flujo de trabajo y trazabilidad

Trello fija alcance, responsables y estado; OpenSpec versiona requisitos y escenarios; GitHub conserva código y revisiones. Las ramas cortas se integran en develop mediante pull requests; main contiene versiones estables. [8]

Las funciones de liderazgo, trabajo en pareja, revisión y validación se describen en el apartado C; los dos últimos corresponden a personas distintas del autor. Varias integraciones finales usaron excepciones autorizadas por plazo, registrando lo no validado. Por ello tienen menos verificación independiente.

F.2. Integración continua

La integración continua se ejecuta en GitHub Actions con cada envío de rama, con cada pull request y bajo demanda; el despliegue continuo la reutiliza como requisito. El frontend usa un archivo de bloqueo de versiones y el ciclo AWS tiene dependencias fijadas específicas; los servicios Python conservan también dependencias con versiones mínimas, por lo que no existe una resolución exacta global. Se comprueban trazabilidad, higiene, especificaciones, pruebas de los tres servicios Python, estilo, pruebas, construcción y tipos del frontend, y la página pública. Tres trabajos de ámbito AWS comprueban PostgreSQL, la monitorización aislada y el controlador con Terraform, sin conexión a la nube. La política de revisión se comprueba aparte. [13]

La CI comprueba estáticamente la topología Docker, sin arrancar el producto completo. También levanta PostgreSQL y pgvector reales para contratos de aplicación, y contenedores desechables de Grafana y Prometheus para probar cifrado, autenticación, paneles y persistencia tras reinicio. Otras integraciones optativas se omiten y se documentan en ensayos aparte. Estas pruebas no invocan AWS ni certifican el despliegue completo. [13]

Tabla F.1. Piezas DevOps de la versión final y alcance de su evidencia

Pieza	Qué hace	Qué acredita y qué no
Trazabilidad	Un identificador de tarea común a Trello, OpenSpec (especificaciones versionadas) y GitHub	Une alcance, requisitos, código y revisiones; una asignación no prueba participación.
Integración continua	10 ejecuciones técnicas, incluida la matriz Python, por cambio y política de revisión en las pull requests	Que el código compila y pasa sus pruebas; no arranca el sistema completo.
Contenedores	Docker Compose: 11 servicios permanentes, incluida la página pública, y 2 de inicialización; perfiles opcionales de IA y demostración	Entorno reproducible con Docker; no es una topología de producción.
Despliegue continuo privado	Agente en el servidor propio: cada 5 minutos busca una versión elegible y la promueve tras 5 comprobaciones	Despliegue real en validación, con datos desechables y proveedores simulados.
Ciclo en AWS	2 pipelines manuales: creación y retirada de un entorno desechable	Código y pruebas sin conexión; nada desplegado en AWS en la versión final.
Observabilidad	Prometheus cada 15 s; paneles y una alerta en Grafana	Diagnóstico técnico; sin receptor externo de alertas ni objetivos de servicio demostrados.

F.3. Contenedores y entorno local

Cuatro Dockerfiles empaquetan frontend, backend, processor y API de costes simulada. La página pública se empaqueta por separado. Docker Compose los conecta con CockroachDB, RabbitMQ, PostgreSQL con pgvector, MinIO, Prometheus y Grafana. Las imágenes base están fijadas por su huella (digest); las aplicaciones se ejecutan sin privilegios de administrador y con comprobaciones de salud. El gateway de modelos de IA y su base de datos forman un perfil opcional, de modo que el sistema arranca completo sin credenciales de un proveedor externo. [4]

Dos ensayos documentan el entorno: uno construyó el sistema y pasó la prueba de humo —un recorrido corto por salud, autenticación, ingesta y procesamiento— en el servidor del equipo; otro repitió la prueba en Docker Desktop, con un control negativo y la persistencia de datos tras reiniciar. Son entornos de ensayo, no una certificación de producción.

F.4. Despliegue continuo

Una CI correcta declara elegible una versión de develop. El agente del servidor prepara una copia aislada y prueba salud, login, ingesta, facturación y cola antes de promoverla. Si falla, reactiva la versión anterior y sus volúmenes; no revierte migraciones ni restaura los datos retirados. El entorno usa datos desechables y proveedores simulados, con promociones registradas. [4]

F.5. Observabilidad

Los logs JSON incluyen un identificador que acompaña a la petición y a la ingesta en cola. Prometheus recoge métricas y Grafana muestra latencias, errores, ingestas y consultas. La alerta de fallos de ingesta cuenta reintentos y no tiene receptor externo. [4]

Un segundo panel de Grafana reúne los resultados de la evaluación del asistente y los indicadores de la memoria, con el modelo declarado en cada ejecución. Su regeneración se describe en el apartado H.1. [20]

F.6. Documentación y decisiones

Los manuales describen la operación, las decisiones de arquitectura justifican alternativas y los informes conservan evidencia. Integrar una decisión no la ratifica. [7]

La memoria tiene además un repositorio independiente. Su flujo de GitHub Actions compila el documento y la bibliografía con LuaLaTeX y biber, adjunta el PDF a las pull requests y, al integrar cambios en main, guarda la versión declarada si todavía no existe. Automatiza la entrega documental de forma independiente del despliegue de la aplicación. [21]

Tabla G.1. Configuración de la versión final y evidencias de la elección

Ámbito	Dato o elección	Qué cuenta
Modelo de chat por defecto	DeepSeek V4.1 Flash; proveedor DeepInfra, 8 bits; sin ruta de reserva	Único generador; no certifica qué modelo respondió en cada llamada.
Modelo de embeddings	text-embedding-3-small	Configurado en las evaluaciones; el ejemplo local usa embeddings simulados.
Alternativa nativa en AWS	Amazon Nova Micro y Titan V2	Probada sin conexión; apagada por defecto, sin invocaciones reales.
Comparativa de contenido	27 de 28 respuestas aceptables con cada modelo	GLM-5.2 y Flash con las mismas preguntas; lo valora un juez IA (otro modelo de lenguaje).
Comparativa de coste	GLM-5.2: 0,1806 USD; Flash: 0,0392 USD	28 operaciones por modelo, con los embeddings de las preguntas y sin la indexación inicial.
Coste facturado medio	GLM-5.2: 0,0026 USD; Flash: 0,0006 USD	GLM-5.2, media por consulta de chat terminada (100 consultas); Flash, media por llamada al modelo (73 llamadas). Son ventanas y unidades distintas, no comparables entre sí.
Coste histórico por operación	Flash: unos 0,0011 USD	Operaciones completas de la campaña histórica; informado por el servidor, no facturado por operación.
Latencia en el entorno desplegado	Percentil 95 por repetición: de 63,0 a 238,1 s (6 repeticiones); objetivo de 10 s	Hasta el estado terminal, con reintentos y fallos, en DockerDevelop (entorno de desarrollo desplegado en un servidor Docker compartido); no es una sola llamada.
Latencia histórica del banco	Percentil 95: 41,3 s antes del arreglo y 42,4 s tras él	Medida en un ordenador local, sobre 28 operaciones completadas antes del arreglo y 27 después.

G. Uso de modelos preentrenados

Economicon no entrena modelos propios: usa modelos preentrenados a través de un gateway y los guía con instrucciones y contexto. La elección importa porque determina el coste por consulta, el tiempo de respuesta y dónde viajan los datos del cliente. Este apartado explica por qué se descartó entrenar, por qué ruta se accede a los modelos, cómo se eligió el modelo y qué se puede afirmar sobre su coste y su latencia. La tabla G.1 resume los datos clave. [5, 6, 12, 14, 22]

G.1. Adaptación sin entrenamiento propio

Para un modelo preentrenado caben tres opciones: entrenarlo desde cero, ajustar sus pesos con datos propios (fine-tuning) o usarlo tal cual con instrucciones y contexto. Para el MVP se eligió la tercera, que concentra el esfuerzo en lo que diferencia a Economicon y evita mantener un ciclo propio de entrenamiento: las fuentes, los controles de la respuesta y el cálculo determinista de las cifras. El modelo de embeddings también se usa sin modificar. [14]

G.2. Ruta y condiciones de uso

Las peticiones al modelo pasan por el gateway LiteLLM y por OpenRouter, que las encamina al proveedor (apartado D). La aplicación solo conoce un alias estable, de modo que cambiar de modelo es un cambio de configuración y no de código. [14]

Como los datos del cliente viajan al proveedor en cada consulta, la configuración exige retención cero de datos (ZDR, Zero Data Retention), excluye a los proveedores que recopilan los datos de las peticiones y desactiva el cambio automático de proveedor ante un fallo. La cuantización a 8 bits representa los parámetros del modelo con menor precisión para abaratar la inferencia. Son condiciones configuradas, no una atestación externa de cada llamada.

Para el ensayo en AWS se implementó una ruta alternativa con los modelos nativos de la nube, sin pasar por el gateway. El código y sus controles están probados sin conexión, pero no se ha invocado ningún modelo de AWS, así que esta memoria no aporta mediciones de esa ruta. [12]

G.3. Cómo se eligió el modelo

GLM-5.2 fue el primer modelo del chat. Para sustituirlo se compararon GLM-5.2 y DeepSeek V4.1 Flash con las mismas preguntas, el mismo corpus y el mismo contexto. Ambos obtuvieron casi todas las respuestas aceptables según el juez IA; Flash costó menos de la cuarta parte; GLM-5.2 necesitó menos reintentos y tuvo tiempos más regulares. Pesó el coste, porque la calidad observada era equivalente. El coste facturado mide llamadas al modelo, no conversaciones completas. Proyectar el gasto mensual exige contar las dos fases por intento, reintentos y embeddings; la tabla G.1 separa esas unidades y sus fuentes. La latencia es el límite principal medido del capítulo. [6] La valoración del juez describe aquella muestra y no sustituye una validación humana. [22]

Las evaluaciones posteriores usaron otros procedimientos, separados en el apartado H: reglas automáticas con una revisora humana y, en las últimas repeticiones, un único juez IA. Sus puntuaciones no se suman ni demuestran superioridad general de un modelo, y la ventaja de coste procede de ventanas de medición distintas, por lo que no se atribuye solo al modelo. [5]

G.4. Latencia y margen de mejora

Las latencias de la tabla G.1 quedan por encima del objetivo. La histórica se midió en local y las repeticiones posteriores en DockerDevelop, ya con el límite final ampliado, se alejan más aún, y sus tiempos incluyen reintentos y operaciones fallidas; ambas se recogen por separado en el apartado H. Falta una comparación controlada que aísle el efecto del límite, del modelo o de la infraestructura. [5]

Las vías de ajuste, todavía sin evaluar, se recogen en el apartado I.

H. Métricas y evaluación de la solución

Un asistente que redacta sobre costes no se puede dar por bueno por inspección. Cada afirmación de esta memoria sobre su calidad, su tiempo o su coste procede de una medición con definición y población conocidas. Este apartado explica qué se mide y cómo se consulta en Grafana, y después presenta cuatro mensajes que esas mediciones sostienen: lo que funciona, el margen de mejora en las respuestas, el tiempo y el coste. Las tablas H.1 y H.2 recogen las cifras.

H.1. Qué se mide y cómo se ve

Las métricas se agrupan en seis familias, cada una con la pregunta a la que responde: exactitud (¿es correcta la respuesta?), relevancia de la recuperación (¿procede del documento y la sección que tocan?), fundamento (¿las citas y las cifras se apoyan en lo recuperado?), latencia (¿cuánto tarda?), robustez de la salida (¿cumple el esquema?) y disponibilidad (¿qué parte de las peticiones no acaba en error?). A ellas se suman tres de negocio —tiempo ahorrado, cobertura del gasto asignado y ahorro potencial— cuyos objetivos son hipótesis de piloto. [20]

Un catálogo único fija la fórmula, la población y la fuente de cada métrica, y se rechaza un resultado calculado con una definición anterior. Las tasas se publican con su base (k de n) y no se promedian entre indicadores. Los casos bloqueados por un fallo de infraestructura se cuentan aparte, para que un fallo del entorno no baje la exactitud ni se oculte. Con 28 casos uno solo pesa 3,6 puntos, de modo que las diferencias pequeñas entre cortes no son concluyentes.

El banco contiene 28 preguntas: 20 con respuesta, 6 que exigen pedir información y 2 que requieren

abstenerse. Un caso aprueba solo si cumple todas sus comprobaciones, entre reglas automáticas y juicios sobre la respuesta. La robustez añade 29 casos, 13 con datos intactos y 16 con defectos en etiquetas, reparto de costes, indicadores, ámbito o anomalías, en los que se exige identificar el defecto sin inventar un resultado. Los objetivos son aprobar el 80 % de las preguntas con respuesta, cumplir el 90 % de las reglas objetivas, no dar cifras sin rastro en casos críticos y terminar el 95 % de las conversaciones en diez segundos. El histórico usó un ordenador local, chat generativo, cuatro documentos y datos sintéticos, con una pasada y una revisora por campaña; las repeticiones posteriores se midieron en DockerDevelop (apartado G) con un único juez IA. [5, 7]

Todo se consulta en un único panel de Grafana, «Asistente y evidencia», con cinco secciones: negocio y fiabilidad de los datos, calidad y robustez, recuperación y citas, coste y rendimiento, y salud técnica. Reúne los paneles anteriores de evaluación y de métricas técnicas, que se conservan. Los resultados de las campañas salen de informes versionados cargados en PostgreSQL y las señales en vivo, de Prometheus; cada dato muestra el modelo declarado y su procedencia. Reproducir el panel no equivale a una evaluación semántica independiente, porque no todas las respuestas y juicios privados están publicados.

H.2. Lo que funciona

Lo que calcula el código determinista funciona y mejoró de forma medible (tabla H.1). Con el arreglo, el documento correcto se recuperó en 19 de 19 casos evaluables, frente a 8 de 20 antes, aunque una captura distinta impide la comparación directa. Las citas apuntaban siempre a un fragmento recuperado, sin certificar su corrección semántica, y no hubo cifras sin rastro en los casos críticos. No hubo errores de servidor en el banco histórico, aunque una operación quedó bloqueada. [5]

Tabla H.1. Banco de 28 preguntas en tres cortes

Qué cuenta	Plantilla sin modelo	Flash, antes del arreglo	Flash, tras el arreglo	Objetivo
Preguntas con respuesta aprobadas	0 de 20	6 de 20	7 de 19	80 %: por alcanzar
Comprobaciones objetivas cumplidas	26 de 51	31 de 51 (60,8 %)	28 de 46 (60,9 %)	90 %: por alcanzar
Comprobaciones juzgadas cumplidas	—	25 de 30 (83,3 %)	26 de 28 (92,9 %)	—
Citas que apuntan a un fragmento recuperado	—	22 de 22	17 de 17	—
Cifras sin rastro	—	1 (no crítica)	0	0 en críticos: se cumple
Documento correcto recuperado	—	8 de 20	19 de 19	—

El detector determinista de anomalías marcó 5 episodios y los 5 eran eventos reales; encontró los 5 eventos detectables de los 7 inyectados, porque los otros dos no lo son por diseño. El retraso hasta la detección fue de 0 días, esperable por la construcción del conjunto, y la reconciliación de los datos de coste no mostró diferencias en 3 pares cliente-divisa. Son datos sintéticos: demuestran coherencia interna y no la integridad de facturas reales. El detector no detecta picos desde cero ni servicios nuevos, porque exige una semana de referencia positiva.

H.3. Margen de mejora en las respuestas

Ningún corte histórico fue aceptado. Tras el arreglo se aprobaron 7 de las 19 preguntas con respuesta y se cumplió el 60,9 % de las comprobaciones objetivas, lejos del 80 % y del 90 % previstos; la plantilla sin modelo no había aprobado ninguna pregunta. Aprobar comprobaciones sueltas es menos exigente que aprobar casos completos, por lo que las valoraciones juzgadas son mejores que las objetivas. En las repeticiones posteriores, los casos conformes variaron entre 7 y 13 por ejecución (tabla H.2). La comparación por juez IA del apartado G.3 empleó otro método y no sustituye estos resultados. [5]

Antes del arreglo, la mitad de los casos fallidos lo fue solo por el lector numérico, aun con cuentas correctas. El lector produce falsos negativos conocidos, cuyo efecto en el corte posterior no se ha cuantificado.

En la batería de robustez de 29 casos, aplicada con el modelo anterior y con Flash antes y después del arreglo, se aprobaron 7, 7 y 15 casos, es decir, el 29,2 %, el 25,0 % y el 60,0 % de los evaluados, con 5, 1 y 4 casos bloqueados. Las prohibiciones concretas evaluadas se cumplieron en los tres cortes. Tras el arreglo se resolvieron reparto y ámbito; etiquetas e indicadores fueron parciales y anomalías quedó bloqueado.

En el ensayo histórico, el arreglo descrito en el apartado E.4 redujo los cortes de respuesta: en 14 preguntas repetidas cuatro veces, bajaron de más de un tercio a uno de 56, y persistían frases incompletas al omitir cifras. Con el límite final ampliado a 10.000 tokens, una demo separada cumplió cuatro de cinco rúbricas con aceptación desfavorable: faltó investigación por servicio y recurso y apareció una afirmación global sin apoyo. Fue una revisión automática por un agente de IA, sin validación humana independiente. El código posterior incorpora cifras de pantalla citables y fuentes agrupadas al final; los cortes de flujo siguen abiertos. Las tablas conservan sus mediciones históricas. [23]

H.4. El tiempo de respuesta

El tiempo es el mayor déficit medido. El percentil 95 histórico fue de 41,3 s antes del arreglo y de 42,4 s tras él, sobre 28 y 27 operaciones completadas, más de cuatro veces el objetivo de 10 s. En las repeticiones posteriores en DockerDevelop (tabla H.2) se alejó más: entre 63 y 238 s por repetición. Las 6 repeticiones previeron 171 operaciones, de las que 158 dieron respuesta y 13 quedaron bloqueadas. [5]

Tabla H.2. Repeticiones posteriores: resultados provisionales por ejecución

Batería y repetición	Respuestas / previstas	Casos conformes / fallidos / bloqueados	Percentil 95 total (s)
Banco 28 · 1	27 / 28	13 / 14 / 1	144,177
Banco 28 · 2	25 / 28	8 / 17 / 3	148,796
Banco 28 · 3	25 / 28	12 / 13 / 3	151,506
Robustez 29 · 1	27 / 29	11 / 16 / 2	106,866
Robustez 29 · 2	26 / 29	9 / 17 / 3	238,114
Robustez 29 · 3	28 / 29	7 / 21 / 1	62,960

Los tiempos llegan hasta el estado terminal, incluyen reintentos y operaciones fallidas y excluyen la espera de conciliación contable. No son duraciones de una sola llamada ni se promedian para obtener un percentil 95 conjunto, y tampoco se descomponen por etapa de forma fiable. Los estados de caso combinan el juicio de un único juez IA con el lector numérico original y sus falsos negativos conocidos; no son una puntuación semántica pura y ninguna campaña está aceptada.

H.5. El coste de la inteligencia artificial

El coste de los modelos no es el factor limitante. La tabla G.1 separa llamadas y operaciones. En la ventana contrastada, de 75 llamadas (73 Flash y 2 GLM), la estimación y la factura difirieron menos de un 1 %; no es una factura por conversación. Las seis repeticiones posteriores sumaron unos 0,53 USD en el registro liquidado del gateway, con embeddings y reintentos incluidos; no es factura ni gasto acumulado del proyecto. [5, 6]

H.6. Qué se puede afirmar y qué medir después

Las mediciones demuestran recorridos observados y trazabilidad de citas en una muestra pequeña. La demo sigue clasificada con incidencias y las seis repeticiones continúan provisionales, con bloqueos y un único juez IA. No acreditan ausencia general de cifras inventadas, superioridad de un modelo, mejora causal del arreglo ni comportamiento con clientes reales. [5]

El propio panel señala lo que falta por medir: el juicio humano independiente en las repeticiones recientes, los tiempos por etapa y HTTP, las distancias de recuperación, el comportamiento con datos de una organización real y la disponibilidad con una referencia anterior con la que comparar. Las métricas de negocio tienen definiciones y un ejemplo reproducible, pero faltan un piloto y beneficios medidos. El resto de las vías de mejora figura en el apartado I. [20]

I. Conclusiones y trabajo futuro

I.1. Logros frente a los objetivos

Economicon implementa ingesta y normalización de gasto, indicadores, anomalías y conversación documental, con separación por cliente, construcción y despliegue automatizados. El servidor calcula los agregados; la generación puede contener errores, como muestra h. Grafana presenta informes con procedencia y límites.

El alcance demostrado usa datos públicos y sintéticos y una API Azure simulada. Recuperación híbrida, chat generativo y ciclo AWS están implementados, pero requieren activación o ejecución específica. Su uso con una organización real no está probado.

I.2. Lecciones y límites

Superar pruebas técnicas no basta para aceptar el asistente. Hay citas trazables, pero también fallos de criterio, bloqueos y latencia excesiva.

El histórico y las seis repeticiones usan métodos distintos y cambiar varias condiciones a la vez impide aislar causas. integrar por excepción tampoco equivale a revisión cruzada completa.

I.3. Trabajo futuro

Validación con una organización real

Se probará con usuarios y datos reales, comprobando permisos, aislamiento e impacto económico. Las repeticiones existentes se complementarán con revisión humana independiente y mediciones en el entorno elegido, variando una condición cada vez.

Herramientas para el modelo y tiempo de respuesta

Se ofrecerán al modelo consultas autorizadas de gasto, propiedad y anomalías, sin generar SQL. Para reducir latencia se estudiarán salidas acotadas sin truncado, caché de contexto y recuperación híbrida con su índice preparado.

Una referencia predictiva para las anomalías

La regla actual compara dos semanas: exige un aumento superior al 20 % y a 100 unidades monetarias diarias. No distingue crecimiento normal de salto inesperado. Se comparará regresión lineal con modelos estacionales y no supervisados, usando un histórico de semanas o meses todavía inexistente y evaluando precisión y explicabilidad. [24]

Detección periódica y avisos

La detección actual es manual y guarda episodios por cliente, moneda y servicio. Se añadirá ejecución periódica, apagada por defecto y coordinada con la carga de costes. Antes de avisar en la aplicación o Grafana se definirán frecuencia, duplicados y condiciones de envío.

Despliegue en la nube

El ciclo desechable en AWS está preparado. Ejecutarlo permitirá medir la ruta con modelos nativos y comparar coste y latencia con la configuración actual.

Referencias

[1] Gartner. *Gartner Forecasts Worldwide Public Cloud End-User Spending to Total $723 Billion in 2025*. 19 de nov. de 2024. URL: https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025.

[2] Flexera. *State of the Cloud Report 2025*. 2025. URL: https://info.flexera.com/CM-REPORT-State-of-the-Cloud-AWS.

[3] *Arquitectura técnica de la solución y guía de ingesta documental*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/architecture.md.

[4] *Despliegue con Docker Compose, integración continua, despliegue continuo y observabilidad*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docker-compose.yml.

[5] *Informes de evaluación y coste del asistente: bancos, robustez, repeticiones, demo y facturación*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/develop/docs/evidence.

[6] *Coste por llamada y operación*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/validation/model-change-memory-figures.md.

[7] *Registro de decisiones de arquitectura*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/develop/docs/adr.

[8] *Gobernanza, contribuciones e índice de continuidad del proyecto*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/CONTRIBUTING.md.

[9] *Observatorio del equipo: instantánea de Trello del 23/10/2026*. 126 historias de trabajo; huella SHA-256 canónica b314f2d749fc5f4717c0a3fda5ac8434691eaca262432c082b9e05b615c376b9. Archivo fuera del repositorio. 23 de oct. de 2026.

[10] *Propuestas integradas y coautoría en la rama principal de tfm-economicon*. 23 de oct. de 2026. URL: https://github.com/EconomiconFinOps/tfm-economicon/commits/develop.

[11] *Conjunto de datos de Azure: fuente y licencia*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/data/azure-sample-dataset.md.

[12] *AWS: monitorización y ciclo de defensa*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/deployment/aws-defensa.md.

[13] *CI: datos y monitorización*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/.github/workflows/ci.yml.

[14] *Configuración de referencia del gateway de modelos*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/infra/litellm/config.example.yaml.

[15] *Especificaciones OpenSpec del asistente*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/develop/openspec/specs.

[16] *Informe de campañas RAG A–E*. Documento en Google Drive. 17 de oct. de 2026. URL: https://drive.google.com/file/d/12hQ7bGLJ17aVbHofBy1Rikg4DMlR8eV5/view.

[17] *Selección y contexto del chat*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/apps/backend/app/services/chat_context.py.

[18] *Prompt del chat: mensaje de sistema, plantilla de entrada y manifiesto*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/develop/apps/processor/app/agents/prompt_resources/jup035/chat/v4.

[19] *Código del chat: validación de la respuesta y reintentos*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/apps/processor/app/clients/chat_gateway.py.

[20] *Panel de Grafana, catálogo de métricas y guías de evaluación*. URL: https://github.com/EconomiconFinOps/tfm-economicon/tree/develop/docs/validation.

[21] *Compilación y entrega de la memoria*. URL: https://github.com/EconomiconFinOps/memoria-economicon/blob/main/.github/workflows/memoria.yml.

[22] *Comparativa de GLM-5.2 y DeepSeek V4.1 Flash*. Documento en Google Drive. 19 de oct. de 2026. URL: https://drive.google.com/file/d/1e_X5dpDBifJrDUDKGyy9HeXpdNeBpSv3/view.

[23] *Salida del chat y límites*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/docs/continuidad/respuestas-largas-chat.md.

[24] *Regla de detección de anomalías y episodios por cliente, moneda y servicio*. URL: https://github.com/EconomiconFinOps/tfm-economicon/blob/develop/apps/backend/app/services/anomaly_episodes.py.